

Bagian 1: Analisis & Desain

a. Ruang & Distribusi Sort

- Implementasi standar Radix Sort umumnya mengharuskan pembuatan *bins* atau antrean (seperti 10 antrean/queues) secara dinamis per digit untuk mendistribusikan kunci. Implementasi struktur alokasi *queues* ini memakan ruang overhead yang proporsional dengan jumlah elemen memori atau minimum $O(n)$, yang jelas akan melanggar batasan ketat Modul A untuk selalu menekan alokasi tambahan hanya di bawah rentang memori konstan $O(1)$ di luar himpunan array awal.
- Mekanisme tmpArray tunggal pada *Improved Merge Sort* bekerja dengan menukar teknik partisi standar (`slice[:]`) dengan metode indeks partisi sublist maya (*virtual sublists*) (`first`, `mid`, `last`) dan mendaur ulang satu ukuran blok *temporary array* yang dialokasikan satu kali saja sebelum iterasi rekursi dimulai. Hasilnya, setiap pergerakan memori dilakukan antar variabel indeks belaka (tidak memakan memori fisik tambahan di tengah stack eksekusi) sehingga overhead ruang eksekusi berhasil ditekan habis-habisan.
- Bila tetap ingin menggunakan teknik dari Radix Sort tanpa melewati $O(1)$, strategi yang paling realistis adalah menerapkan In-Place Radix Sort (menggunakan partisi berbasis bit/pembagian langsung dalam memori yang sama selayaknya logika Quicksort). Dampaknya terhadap waktu, stabilitas urutan berpotensi akan hilang, namun kompleksitas ruang sukses ditekan.

b. Linked List & Pointer Manipulation

- Fungsi `_splitLinkedList()` berhasil menemukan titik tengah cukup dalam satu kali *traversal* dengan metode *fast-slow pointer* menggunakan dua buah referensi penanda: `midPoint` yang bergerak 1 langkah secara sekuensial, serta `curNode` yang bergerak lebih cepat sejauh 2 langkah. Dalam putaran eksekusinya, setiap kali `curNode` mencapai indeks paling akhir dari *linked list* (*null*), pointer `midPoint` secara intuitif otomatis tersisa tepat di bagian titik tengah.
- Fungsi `_mergeLinkedLists()` dengan teknik *dummy node* dan *tail reference* diunggulkan karena penggabungan simpul node dijalankan dengan hanya memodifikasi arah penunjuk alamat dari satu elemen (atribut `.next`). Simpul awal buatan *dummy* mengeliminasi penanganan *special case* secara kondisional ketika akan menautkan *node* pertama, tanpa sama sekali perlu membuat ruang elemen list baru. Kompleksitas ruang pun menurun murni menjadi $O(\log n)$ yang datang hanya dari kedalaman tumpukan memori (*stack*) eksekusi rekursif.

c. Quick Sort Worst-Case & Pivot Strategy

- a. Saat Quick Sort dijalankan pada urutan array terbalik (descending) dengan pivot mengambil nilai data awal (first), pembelahan `partitionSeq()` akan menderita bias ekstrem di mana segmentasi elemen pivot tersebut selalu membelah list menjadi 1 sublist kosong dan 1 sublist berkapasitas elemen $(n-1)$ di satu sisi saja. Tingkat kedalaman rekursinya akan membengkak dari $O(\log n)$ berubah menuju level linear $O(n)$, meledakkan waktu eksekusinya ke level kuadratik lambat senilai $O(n^2)$.
- b. Strategi yang paling *robust* direkomendasikan adalah mencari pivot menggunakan komparasi **Median-of-Three**, yaitu mencari nilai rata-tengah komparatif dari indeks elemen paling awal (first), indeks tepat di bagian tengah (mid), dan elemen bagian akhir (last) untuk diangkat menjadi pivot representatif.
- c. Kelayakan adaptasi di Singly Linked List: Konsep *Median-of-Three* kurang efisien di Singly Linked List. Karena List tidak bersifat *random access* per-indeks ($O(1)$) melainkan harus di-*traverse* perlahan (looping serial bernilai cost $O(n)$) untuk mengetahui ujung last dan mid-nya, strategi pivot ini akan menaikkan performa eksekusi tak berdasar. Disarankan memakai Merge Sort jika targetnya adalah *linked list*.

d. Batas Teoretis & Paradigma Sorting

- Radix sort dapat mencapai linear order $O(dn)$ secara teoretis tanpa mengontradiksi limit teoretis Comparison Sort $\Omega(n \log n)$ dikarenakan sifat esensial dari Radix Sort yang merujuk sebagai algoritma pengurut **Distribution Sort** (tidak mengutamakan perbandingan logis berpasangan melainkan mengorganisir penempatan ember/urutan per masing-masing indeks karakter kunci).
- Asumsi implisit perihal domain kunci k (angka radix/batas opsi basis kunci e.g., rentang nilai ascii atau angka dasar 10 desimal) dan d (banyaknya digit maksimum angka paling besar di elemen) yang membuat algoritma tersebut melampaui limit Comparison sort dikarenakan baik parameter k atau d pada mayoritas kasus masalah dapat dibulatkan sebagai komponen numerik skalar/konstanta statis berbiaya minim sehingga bisa diturunkan secara asimtotik linear hingga setara $O(n)$.